LIVEPEER DOCS

AI-First Documentation Strategy

22-Item Implementation Plan · docs.livepeer.org · github.com/livepeer/docs (docs-v2 branch)

This report covers every concrete action to make docs.livepeer.org genuinely AI-first — not just human-readable documentation that happens to exist online, but a machine-discoverable, agent-executable, crawlable intelligence surface purpose-built for the era of LLM-driven development. Each item includes effort, impact, Mintlify-specific implementation instructions, and caveats.

Priority Matrix

Tier	Items	Notes
Critical Impact + Low Effort (Do Now)	**1, 3, 4, 5, 10, 11, 19, 20**	Ship these immediately. Mostly config, scripts, and frontmatter work.
Critical Impact + Medium Effort (Do Next)	**6, 8, 12, 16, 22**	High structural value. 1-3 days each. Define templates, write content.
High Impact + Low Effort (Quick Wins)	**2, 7, 9, 14, 23, 24, 25, 27**	Low effort for meaningful gains. Schedule in next sprint.
High Impact + Medium/High Effort (Plan)	**13, 15, 17, 18, 21**	Architectural or engineering work. Requires resourcing and coordination.
Medium Impact + Low Effort (Hygiene)	**26, 28**	Glossary aliases and source frontmatter. Low cost, steady compounding value.

Summary Overview — All 28 Items

#	Category	Item	Effort	Impact
1	Crawlability	**Ship a production llms.txt + llms-full.txt**	Low	Critical
2	Crawlability	**Serve every page as clean Markdown via .md endpoints**	Low	High
3	Crawlability	**Generate and serve a sitemap-ai.xml**	Low	High
4	Crawlability	**Publish all OpenAPI specs at canonical stable URLs**	Low	Critical
5	Crawlability	**Add /.well-known/ai-plugin.json manifest**	Low	High
6	Content Structure	**First-class "Agent Task" page section**	Medium	Critical

7	Content Structure	Machine-readable preconditions/postconditions in frontmatter	Low	High
8	Content Structure	Structured Q&A blocks on every major page	Medium	Critical
9	Content Structure	Entity disambiguation anchors and dfn tags	Low	High
10	Content Structure	Version and freshness metadata on every page	Low	High
11	Discovery & Indexing	Build and serve docs-index.json — full structured site map	Low	Critical
12	Discovery & Indexing	JSON-LD structured data on every page	Medium	Critical
13	Discovery & Indexing	Publish a semantic concept graph	Medium	High
14	Discovery & Indexing	Submit to AI-specific indexes and directories	Low	High
15	Agent-Native	Add "paste into AI assistant" context blocks per section	Medium	High
16	Agent-Native	Publish MCP server docs and ready-to-use config	Medium	Critical
17	Agent-Native	Agent-safety metadata on every OpenAPI endpoint	Medium	High
18	Agent-Native	Explicit failure mode and error recovery pages	Medium	High
19	Freshness & Signals	Publish a machine-readable changelog.json	Low	High
20	Freshness & Signals	Page confidence/verification status metadata	Low	Medium
21	Freshness & Signals	Live network-state data endpoint	High	High
22	Freshness & Signals	Publish a training-context.md — the "one file to rule them all"	Medium	Critical
23	Crawlability	Verify and CI-guard agent-discovery HTTP headers	Low	High
24	Content Structure	Promote the existing AI/Agent quickstart from x-pages to active navigation	Low	High
25	Crawlability	Ensure robots.txt exists and references sitemap	Low	Medium
26	Discovery & Indexing	Glossary synonyms/aliases as structured export	Low	Medium
27	Discovery & Indexing	Curated task-to-page map	Low	High
28	Content Structure	Source-of-truth frontmatter field per page	Low	Medium

Detailed Implementation Guide

1. Ship a production llms.txt + llms-full.txt

Category: Crawlability	Effort: Low	Impact: Critical	

Serve machine-readable content maps at predictable URLs: llms.txt (index) and llms-full.txt (full corpus concat). The guidance file exists (tools/ai-rules/llms.txt.information.md) but the files must be live-served.

Implementation Steps

1. Create /public/llms.txt — list every doc page as: # Title\n> Description\nURL
2. Create /public/llms-full.txt — concatenate all .mdx pages as flat Markdown via a build script
3. In mint.json add: "staticFiles": ["llms.txt", "llms-full.txt"]
4. Add a GitHub Action to regenerate llms-full.txt on every merge to main
5. Verify: curl https://docs.livepeer.org/llms.txt returns 200

⚠️ **Caveats**

> *llms-full.txt can get large (>1MB). Add a build step to strip frontmatter and MDX components before concatenating. Mintlify serves /public/* at root automatically.*

2. Serve every page as clean Markdown via .md endpoints

Category: Crawlability	Effort: Low	Impact: High	

Agents and crawlers prefer raw Markdown over HTML. Add a predictable .md URL pattern for every page so tools like Cursor, Claude Projects, and Perplexity can fetch page content directly.

Implementation Steps

1. In mint.json set "redirects" to map /docs/[slug].md -> raw GitHub content
2. Alternatively: build a Cloudflare Worker that proxies /[path].md requests to raw GitHub MDX
3. Use the GitHub raw URL pattern: raw.githubusercontent.com/livepeer/docs/docs-v2/v2/pages/[path].mdx
4. Add a note in llms.txt: "Append .md to any URL for raw Markdown"
5. Add x-robots-tag: follow in response headers for .md routes

⚠️ **Caveats**

> *MDX with JSX components won't render cleanly as pure Markdown. Pre-process files through a remark/rehype pipeline to strip components before serving. Mintlify doesn't natively support .md routes — a Cloudflare Worker or custom redirect layer is needed.*

3. Generate and serve a sitemap-ai.xml

Category: Crawlability	Effort: Low	Impact: High	

An AI-specific sitemap separate from the SEO sitemap, enriched with content type, semantic category, word count, and last-verified date. Lets AI crawlers prioritize intelligently.

Implementation Steps

1. Create tools/scripts/generate-ai-sitemap.js
2. Parse all v2/pages/**/*.mdx frontmatter for: title, description, category, lastVerified
3. Emit XML with custom <ai:contentType>, <ai:wordCount>, <ai:category> elements
4. Output to public/sitemap-ai.xml
5. Add to GitHub Actions build pipeline: runs on push to main
6. Submit URL to Bing Webmaster Tools and any AI indexing services

⚠️ **Caveats**

> *Custom XML namespaces in sitemaps are ignored by Google but consumed by some AI crawlers. Keep a standard sitemap.xml alongside this — don't replace it. Check Mintlify doesn't auto-generate its own sitemap that conflicts.*

4. Publish all OpenAPI specs at canonical stable URLs

Category: Crawlability	Effort: Low	Impact: Critical	

The API specs exist in /api/ but must be served live at stable, well-known URLs so agents can auto-discover and self-configure tool use without reading documentation at all.

Implementation Steps

1. Verify these are live-served (check now): docs.livepeer.org/api/openapi.json, /api/gateway.openapi.yaml, /api/ai-worker.yaml
2. Add a canonical API index at docs.livepeer.org/api/index.json listing all specs with name, URL, version
3. Add Link headers in HTTP responses: Link: </api/openapi.json>; rel="describedby"
4. Register specs on APIs.guru (apis.guru/add-api) and SwaggerHub
5. Add "x-discoverable": true extension to all OpenAPI root objects
6. Document the canonical URLs prominently on the API reference page

⚠️ Caveats

> *Mintlify may not serve files from /api/ automatically — confirm static file routing. If specs are only in the repo, add them to /public/ or configure Mintlify staticFiles.*

5. Add /.well-known/ai-plugin.json manifest

Category: Crawlability	Effort: Low	Impact: High	

A machine-readable manifest at a well-known URL declaring what Livepeer docs cover, available APIs, auth requirements, and rate limits. Pulled by Copilot, Claude Projects, and GPT plugins.

Implementation Steps

1. Create public/.well-known/ai-plugin.json with: name_for_model, description_for_model, api.url (openapi.json), auth type, contact_email
2. Add to public/.well-known/mcp.json for MCP-compatible agents (see item 16)
3. Ensure Mintlify serves /.well-known/* — add staticFiles config if needed
4. Add a robots.txt line: Allow: /.well-known/
5. Test: curl https://docs.livepeer.org/.well-known/ai-plugin.json

⚠️ Caveats

> *The OpenAI "plugin" spec is deprecated but the well-known path is still polled by many agents as a discovery heuristic. Keep the file but also produce a separate mcp.json for newer agent frameworks.*

6. First-class "Agent Task" page section

Category: Content Structure	Effort: Medium	Impact: Critical	

Expand the existing gateway runbook (get-AI-to-setup-the-gateway.mdx) into a dedicated top-level section of structured agent runbooks — pages written for agents to execute end-to-end, not humans to read.

Implementation Steps

1. Create v2/pages/08_agents/ section in navigation
2. Establish a runbook MDX template: Preconditions, Steps (numbered, atomic), Expected Output, Error Recovery, Rollback
3. Write initial runbooks: Setup AI pipeline, Run a Gateway, Stake LPT, Register Orchestrator
4. Add to mint.json navigation group: { "group": "For AI Agents", "pages": [...] }
5. Ensure each page has frontmatter: agent_executable: true, requires_human: false/true

6. Link from main navigation with an ✦ or robot icon to signal agent-first content

⚠️ **Caveats**

> *Agent runbooks need to stay current — stale runbooks cause agent failures. Add a lastVerified frontmatter field and CI check that flags pages older than 90 days.*

7. Machine-readable preconditions/postconditions in frontmatter

Category: Content Structure	**Effort: Low**	**Impact: High**	

Add structured prerequisites and outcomes to frontmatter on every procedural page so agents can parse them before attempting a task — without reading body prose.

Implementation Steps

1. Extend the frontmatter schema in mint.json / content templates
2. Add fields: prerequisites: [wallet, docker, gpu], outcomes: [pipeline_running, endpoint_live]
3. Also add: difficulty: beginner|intermediate|advanced, time_estimate: "15 mins"
4. Update generate-seo.js to validate these fields exist on procedural pages (type: guide/tutorial)
5. Create a Mintlify custom component <Prerequisites /> that renders them visually for humans
6. Expose the data in docs-index.json (see item 11) so agents can filter before fetching

⚠️ **Caveats**

> *Keep prerequisites as flat arrays of strings, not nested objects — simpler for LLM parsing. Avoid prose in these fields.*

8. Structured Q&A blocks on every major page

Category: Content Structure	**Effort: Medium**	**Impact: Critical**	

Embed 3-5 explicit Q&A pairs per page in a machine-readable format. This is the single highest-leverage AEO tactic — these are what Perplexity, SGE, and ChatGPT surface verbatim for relevant queries.

Implementation Steps

1. Create a <QA question="..." answer="..." /> Mintlify component
2. Render as visible accordion for humans, but also emit JSON-LD FAQPage schema (see item 12)
3. Write Q&As for the top 10 highest-traffic pages first, then roll out sitewide
4. Focus questions on: "How do I...", "What is...", "Why does...", "When should I..."
5. Add a CI check: pages with type: guide|reference must have at least 3 <QA> blocks
6. Store Q&A content in frontmatter OR as MDX component — not both (choose one source of truth)

⚠️ **Caveats**

> *Q&A answers must be self-contained (no "see above" references) since answer engines extract them in isolation. Keep answers under 100 words each for best AEO performance.*

9. Entity disambiguation anchors and dfn tags

Category: Content Structure	**Effort: Low**	**Impact: High**	

For every Livepeer-specific term (LPT, Gateway, Orchestrator, Pipeline), add machine-readable definition anchors. Lets LLMs correctly resolve ambiguous references without hallucinating.

Implementation Steps

1. Audit the generated glossary-terms.json — ensure all key entities are present
2. Add <dfn id="gateway"> HTML anchors in glossary page MDX for each term
3. Add entity_type: protocol_component|token|role|concept to each glossary frontmatter entry
4. In the concept graph (item 13), use these IDs as canonical entity references

5. Add a Mintlify custom component <Term id="gateway" /> that renders inline tooltips for humans
6. Ensure the glossary page is listed first in llms.txt for priority crawl

⚠️ **Caveats**

> *Mintlify's built-in tooltip/glossary features may partially handle this — check if they emit proper HTML id attributes before duplicating. The goal is stable, predictable anchor IDs that never change.*

10. Version and freshness metadata on every page

Category: Content Structure	Effort: Low	Impact: High	

Add introduced_in, deprecated_in, last_verified, and status fields to frontmatter. Agents weigh source freshness heavily — explicit dates prevent them from acting on stale information.

Implementation Steps

1. Add frontmatter fields: last_verified: "2025-01-15", status: "current|deprecated|experimental"
2. Add introduced_in: "v3.2" and deprecated_in: "v4.0" where applicable
3. Update generate-seo.js to require last_verified on all non-index pages
4. Add a GitHub Action: weekly PR that flags pages where last_verified > 90 days ago
5. Create a /data/freshness-report.json listing all pages with their verification dates
6. Show a visible "Last verified" badge on pages using Mintlify's built-in updatedAt feature

⚠️ **Caveats**

> *Don't confuse last_verified (human-confirmed accuracy) with git commit date (automated). These are different signals. Only update last_verified when a human has actually read and confirmed the page.*

11. Build and serve docs-index.json — full structured site map

Category: Discovery & Indexing	Effort: Low	Impact: Critical	

A single JSON file listing every page with: URL, title, description, section, tags, entity types, API endpoints referenced, difficulty, and word count. This is what agents load to plan traversal without blind crawling.

Implementation Steps

1. Create tools/scripts/generate-docs-index.js
2. Parse all v2/pages/**/*.mdx, extract frontmatter + headings + code block languages
3. Output: { version, generated, pages: [{ url, title, description, section, tags, entities, apiEndpoints, wordCount, difficulty, lastVerified }] }
4. Serve at docs.livepeer.org/docs-index.json (add to public/)
5. Add to GitHub Actions: regenerates on every merge to main
6. Reference this file in llms.txt and ai-plugin.json as the discovery endpoint

⚠️ **Caveats**

> *Keep the schema stable — agents cache this file. Version the schema with a "schemaVersion" field. Breaking schema changes should increment the version and maintain backwards compatibility for at least one release.*

12. JSON-LD structured data on every page

Category: Discovery & Indexing	Effort: Medium	Impact: Critical	

Embed TechArticle, HowTo, FAQPage, and APIReference JSON-LD schema in every page's <head>. This is the primary mechanism by which Google AI Overviews, Bing Copilot, and Perplexity extract structured answers.

Implementation Steps

1. In Mintlify, use the "head" frontmatter field or a global <Head> component to inject JSON-LD
2. Map page type to schema: type:guide -> HowTo, type:reference -> TechArticle, type:faq -> FAQPage

3. For HowTo: include step-by-step instructions from page headings + prose
4. For FAQPage: pull from <QA> component blocks (item 8)
5. Extend generate-seo.js to emit a jsonLd object per page that Mintlify injects
6. Validate with Google's Rich Results Test and schema.org validator

⚠️ **Caveats**

> *Mintlify's head injection approach needs testing — some frontmatter fields get escaped. Test with a single page first before rolling out sitewide. HowTo schema has a max recommended step count; split long guides if needed.*

13. Publish a semantic concept graph

Category: Discovery & Indexing	**Effort: Medium**	**Impact: High**	

A machine-readable file mapping relationships between Livepeer concepts: "Gateway depends-on Orchestrator", "AI Pipeline uses ai-worker", "LPT is-staked-by Delegator". Agents reasoning about architecture traverse this instead of keyword-matching.

Implementation Steps

1. Create data/concept-graph.json with nodes (entities) and edges (relationships)
2. Node fields: id, label, type, description, docUrl
3. Edge fields: from, to, relationship (depends-on, uses, is-part-of, manages, earns-from)
4. Seed from the glossary-terms.json — cross-reference with architecture docs
5. Serve at docs.livepeer.org/data/concept-graph.json
6. Add a visual rendering using D3/Mermaid for the human-facing architecture page

⚠️ **Caveats**

> *This requires deep domain knowledge to get relationship types right — allocate review time with Livepeer protocol engineers. A wrong relationship is worse than no relationship. Start with ~20 core entities before expanding.*

14. Submit to AI-specific indexes and directories

Category: Discovery & Indexing	**Effort: Low**	**Impact: High**	

Actively submit Livepeer docs and OpenAPI specs to developer AI tools that ingest third-party documentation. This gets Livepeer surfaced in Cursor, Copilot, and Phind without users needing to configure anything.

Implementation Steps

1. Submit OpenAPI specs to APIs.guru: apis.guru/add-api — free, polled by many AI tools
2. Submit to Zuplo's API Hub and RapidAPI for additional distribution
3. Add docs.livepeer.org to Context7 (context7.com) — a dedicated LLM context directory
4. Submit to Mintlify's own LLM directory (they maintain a list of llms.txt sites)
5. Create a README note in /ai-tools/ with all submission status and renewal dates
6. Monitor via Google Search Console for AI Overview appearances

⚠️ **Caveats**

> *Some directories require API key verification or manual review. APIs.guru is the highest leverage — it's polled by Cursor, Continue.dev, and others. Track submission dates; some need annual re-verification.*

15. Add "paste into AI assistant" context blocks per section

Category: Agent-Native	**Effort: Medium**	**Impact: High**	

A collapsible block on key pages containing a compressed, dense context summary purpose-built for pasting into AI coding assistants. Makes your docs the canonical source that gets pasted — not hallucinated summaries.

Implementation Steps

1. Create a <AIContext> Mintlify component that renders a collapsed "Use with AI" block
2. Inside: a copyable text block covering core concepts, key APIs, gotchas, and code patterns for that section
3. Write these for: Developers (AI pipeline), Gateway operators, Orchestrators, SDK reference
4. Include the section's docs URL so the AI assistant can cite and fetch for more
5. Style distinctively (robot icon, different background) to signal its purpose
6. Add to the context blocks in llms-full.txt with an [AI_CONTEXT] section marker

⚠️ **Caveats**

> *These go stale fast. Treat them as a maintained artifact, not a one-time write. Limit to ~500 words each — the goal is dense signal, not comprehensive coverage. Link to full pages for depth.*

16. Publish MCP server docs and ready-to-use config

Category: Agent-Native	Effort: Medium	Impact: Critical	

Write explicit documentation for using Livepeer APIs via MCP (Model Context Protocol) and publish a livepeer-mcp.json that devs drop into Cursor/Claude/Windsurf. The frontier of agent-first developer experience.

Implementation Steps

1. Create v2/pages/03_developers/mcp-integration.mdx covering MCP setup for Livepeer
2. Write or reference an MCP server implementation for the Livepeer API (use FastMCP or similar)
3. Publish mcp-config.json at docs.livepeer.org/.well-known/mcp.json
4. Include copy-paste config blocks for: Cursor (.cursor/mcp.json), Claude Desktop, Windsurf
5. Document which tools the MCP server exposes: create_pipeline, get_streams, transcode_video, etc.
6. Link from the main developer quickstart prominently

⚠️ **Caveats**

> *Requires an actual MCP server implementation — this is engineering work, not just docs. Coordinate with the Livepeer dev team. If an MCP server doesn't exist yet, document the OpenAPI-based approach as a fallback using tools like openapi-mcp-server.*

17. Agent-safety metadata on every OpenAPI endpoint

Category: Agent-Native	Effort: Medium	Impact: High	

Add x-agent-* OpenAPI extensions to every endpoint: use-case, idempotent, side-effects, requires-human-approval. Agents making autonomous API calls need this for safe decision-making. Nobody does this well yet.

Implementation Steps

1. Audit all endpoints in openapi.yaml, gateway.openapi.yaml, ai-worker.yaml
2. Add to each endpoint: x-agent-use-case: "string describing agent task", x-idempotent: true/false
3. Add: x-side-effects: [charges_credits, creates_resource, deletes_data] where applicable
4. Add: x-requires-human-approval: true for any destructive or financial operations
5. Add: x-cost-per-call where relevant (e.g. AI inference endpoints)
6. Document this extension schema in a new page: docs.livepeer.org/api/agent-safety-extensions

⚠️ **Caveats**

> *OpenAPI extensions (x-*) are non-standard and ignored by most validators — that's fine. Some OpenAPI tools may warn about them. Document the extension schema so agents know what to expect. This is forward-looking infrastructure; payoff increases as agent-native tooling matures.*

18. Explicit failure mode and error recovery pages

Category: Agent-Native	Effort: Medium	Impact: High	

Pages written specifically for agent error recovery: "Why your AI pipeline job failed", "Common Gateway setup mistakes", "Orchestrator connection debugging". Agents hitting errors need structured recovery paths, not general troubleshooting prose.

Implementation Steps

1. Audit support channels (Discord, forum) for the top 10 most common errors
2. Write pages using the format: Error message (exact string) -> Cause -> Fix -> Prevention
3. Include error codes as machine-readable anchors: <dfn id="ERR_PIPELINE_TIMEOUT">
4. Add to frontmatter: error_codes: [ERR_PIPELINE_TIMEOUT, ERR_GPU_OOM]
5. Reference in concept-graph.json: error nodes connected to component nodes
6. Group under a top-level "Troubleshooting" section with its own llms.txt entry

⚠️ **Caveats**

> *Error messages change with software versions — these pages need version tagging (see item 10). Work with engineering to get canonical error codes rather than relying on prose descriptions.*

19. Publish a machine-readable changelog.json

Category: Freshness & Signals	**Effort: Low**	**Impact: High**	

A stable JSON endpoint that agents poll to know if their cached knowledge of Livepeer docs is stale. Includes: date, affected pages by URL, change type (breaking/additive/deprecation), and a plain-language summary.

Implementation Steps

1. Create public/changelog.json with schema: { version, entries: [{ date, type, affectedUrls, summary, breakingChange }] }
2. Add a GitHub Action that appends to changelog.json on every merge to main that touches v2/pages/
3. Use git diff to auto-detect which pages changed and classify by commit message convention (feat/fix/breaking)
4. Surface this at docs.livepeer.org/changelog.json and reference in ai-plugin.json
5. Also maintain a human-readable CHANGELOG.md from the same data
6. Add an HTTP Last-Modified header so agents can conditionally fetch

⚠️ **Caveats**

> *Automated classification of change types is imperfect — "breaking" requires human judgment. Add a manual override: a .changelog-override file in the PR that contributors fill out for significant changes.*

20. Page confidence/verification status metadata

Category: Freshness & Signals	**Effort: Low**	**Impact: Medium**	

A frontmatter field (verified, community-maintained, experimental) telling agents how much to trust a page. Agents weigh source confidence — you can actively influence that scoring.

Implementation Steps

1. Add confidence: verified|community|experimental|deprecated to all page frontmatter
2. Map to visible badges via a Mintlify custom component — humans benefit from this signal too
3. Update generate-seo.js to require this field; default to "community" if absent
4. In docs-index.json (item 11), include confidence per page entry
5. In llms.txt, prefix community-maintained pages with a [community] tag
6. Add a CI gate: pages with confidence: verified require a reviewer in CODEOWNERS before merge

⚠️ **Caveats**

> *Avoid a binary verified/not-verified — use the three-tier system to allow community contributions without degrading overall trust signals. Tie the "verified" gate to the existing CODEOWNERS system to avoid extra process overhead.*

21. Live network-state data endpoint

Category: Freshness & Signals	Effort: High	Impact: High	

A live JSON endpoint at docs.livepeer.org/data/network-state.json with current pipeline availability, active orchestrators, and LPT price. Agents building Livepeer integrations need this at runtime — bridges docs and live data.

Implementation Steps

1. Create a Cloudflare Worker or serverless function that queries the Livepeer API + CoinGecko
2. Output: { timestamp, lptPrice, activePipelines, activeOrchestrators, networkStatus, apiHealth }
3. Cache with a 60-second TTL to avoid hammering upstream APIs
4. Serve at docs.livepeer.org/data/network-state.json with proper CORS headers
5. Document the endpoint schema on a dedicated page in the docs
6. Reference in ai-plugin.json as a "live context" endpoint separate from static documentation

⚠️ Caveats

> *This is infrastructure work, not just docs work. Requires agreement on what data to expose and who maintains the worker. Start with a static fallback that returns a "data unavailable" response rather than a 404 if the live fetch fails.*

22. Publish a training-context.md — the "one file to rule them all"

Category: Freshness & Signals	Effort: Medium	Impact: Critical	

A single, maximally dense Markdown file covering the full Livepeer conceptual model, API surface, common patterns, and gotchas — purpose-built for developers to paste into AI system prompts or RAG pipelines. Nobody ships this explicitly; it's a major differentiator.

Implementation Steps

1. Create ai-tools/livepeer-training-context.md — NOT inside v2/pages/, it's a standalone artifact
2. Structure: Protocol Overview -> Key Entities -> API Quick Reference -> Common Patterns -> Known Gotchas
3. Target length: 2,000-4,000 words — dense signal, not comprehensive coverage
4. Include canonical API endpoints, key parameters, authentication pattern, and error codes
5. Serve at docs.livepeer.org/ai-tools/training-context.md and reference from llms.txt as top entry
6. Establish a quarterly review process — this file needs to stay accurate above all others

⚠️ Caveats

> *This file requires deep expertise to write well — it's not auto-generatable. Budget significant engineering review time. Don't try to cover everything; ruthlessly prioritize the 20% of concepts that appear in 80% of developer queries.*

23. Verify and CI-guard agent-discovery HTTP headers

Category: Crawlability	Effort: Low	Impact: High	

Mintlify emits Link and X-Llms-Txt response headers pointing to /llms.txt on every page. These headers are how agents auto-discover the LLM index without prior knowledge of the site structure. Currently unverified and unguarded — a silent regression risk.

Implementation Steps

1. Verify headers are present now: curl -I https://docs.livepeer.org | grep -i llms
2. Expected: Link: </llms.txt>; rel="llms-txt" and X-Llms-Txt: https://docs.livepeer.org/llms.txt
3. If missing: check Mintlify docs for the ai.llmsTxt config option in docs.json and enable it
4. Add a CI smoke test in .github/workflows/ that curls the deploy preview URL and asserts both headers are present (fail the check if either is absent)
5. Also assert /llms.txt returns HTTP 200 and /llms-full.txt returns HTTP 200 in the same check
6. Document expected headers in ai-tools/ so future maintainers know they exist and why

⚠️ Caveats

> *Mintlify controls these headers server-side — you cannot set them via static files. If the headers are absent, the fix is in docs.json config or a Mintlify support request, not in the repo. Pin the CI check to run on every deploy preview, not just production merges.*

24. Promote the existing AI/Agent quickstart from x-pages to active navigation

Category: Content Structure	Effort: Low	Impact: High	

A page already exists at v2/x-pages/07_resources/documentation-guide/docs-features-and-ai-integrations.mdx but is not in the active page tree or docs.json navigation. It needs moving and wiring — zero content work required, just promotion.

Implementation Steps

1. Move or copy the file to v2/pages/07_resources/documentation-guide/docs-features-and-ai-integrations.mdx
2. Open docs.json and add the route to the appropriate navigation group (Resources or a new "For AI Agents" group)
3. Review the page content and update any stale links or references to x-pages paths
4. Add frontmatter: title, description, persona: developer, tags: [ai, agents, llm]
5. Link to it from the main introduction page and the developer quickstart as a "Using AI with Livepeer docs" reference
6. Once promoted, ensure it appears in llms.txt and the docs-index.json (item 11)

⚠️ Caveats

> *Check why the page was in x-pages — it may have been deliberately staged. Confirm it is complete and accurate before promotion. x-pages is typically a holding area, so content quality review is needed before making it a top-level navigation item.*

25. Ensure robots.txt exists and references sitemap

Category: Crawlability	Effort: Low	Impact: Medium	

A robots.txt at the site root is how crawlers discover crawl scope and find the sitemap. Without it, crawlers fall back to defaults and may miss or incorrectly restrict AI indexing. Mintlify may auto-generate one — it needs verification.

Implementation Steps

1. Check if robots.txt exists: curl https://docs.livepeer.org/robots.txt
2. If Mintlify auto-generates it, verify it contains: Sitemap: https://docs.livepeer.org/sitemap.xml
3. Also verify it does NOT accidentally disallow AI crawlers (some templates block GPTBot, ClaudeBot, etc.)
4. If custom robots.txt is needed, add to public/robots.txt with: User-agent: * Allow: / Sitemap: https://docs.livepeer.org/sitemap.xml
5. Explicitly allow known AI crawlers: User-agent: GPTBot Allow: / and User-agent: ClaudeBot Allow: /
6. Add robots.txt to the CI smoke test (item 23) — assert it returns 200 and contains the Sitemap directive

⚠️ Caveats

> *Mintlify may own robots.txt and not allow custom overrides via the public/ directory. Check Mintlify docs for the robotsTxt config option before creating a file manually. Conflicting robots.txt files (repo + Mintlify generated) can cause unpredictable crawl behaviour.*

26. Glossary synonyms/aliases as structured export

Category: Discovery & Indexing	Effort: Low	Impact: Medium	

Extend the existing generate-glossary.js to include synonyms and aliases per term, and export the full glossary as a structured JSON artifact for retrieval pipelines. Currently the glossary is generated but not alias-aware — agents querying "transcoder" won't resolve to "Orchestrator".

Implementation Steps

1. Open tools/scripts/snippets/generate-data/scripts/generate-glossary.js

2. Extend the term schema: add aliases: string[] and synonyms: string[] fields
3. Populate known aliases for core terms: Orchestrator (transcoder, node), Gateway (broadcaster), LPT (Livepeer Token), Pipeline (AI pipeline, workflow)
4. Export the enriched glossary as snippets/data/glossary.json with the full alias map
5. Serve at docs.livepeer.org/data/glossary.json (add to public/ or static assets)
6. Reference in docs-index.json (item 11) and llms.txt as a "terminology" resource entry

⚠️ **Caveats**

> *Synonym curation requires domain expertise — do not auto-generate synonyms with an LLM without human review, as incorrect aliases (e.g. mapping "Gateway" to a generic load balancer concept) will cause agent mis-routing. Keep a changelog for the glossary data file.*

27. Curated task-to-page map

Category: Discovery & Indexing	Effort: Low	Impact: High	

A structured "task → best page(s)" lookup: "run a gateway" → /gateways/quickstart, "use Studio API" → /developers/studio-api, "stake LPT" → /delegators/staking. Agents routing user intent need this to avoid full-corpus search for common tasks.

Implementation Steps

1. Create snippets/data/task-map.json with schema: { tasks: [{ intent: string, keywords: string[], primaryUrl: string, secondaryUrls: string[], persona: string }] }
2. Seed with 20–30 top developer and operator tasks derived from Discord/forum support queries
3. Serve at docs.livepeer.org/data/task-map.json
4. Reference from the AI/Agent quickstart page (item 24) with a human-readable rendering
5. Include in ai-plugin.json as a "task routing" resource so agents discover it via the manifest
6. Add a CI check: all primaryUrl values must resolve to real pages (crawl the sitemap to validate)

⚠️ **Caveats**

> *The task map goes stale as page URLs change. The CI URL validation check (step 6) is essential. Use stable slug paths, not deep nested URLs, to reduce maintenance burden. Limit to genuinely high-frequency tasks — a bloated map with 200 entries loses signal.*

28. Source-of-truth frontmatter field per page

Category: Content Structure	Effort: Low	Impact: Medium	

Add a source frontmatter field on each page pointing to the canonical upstream: a GitHub file, OpenAPI spec path, or spec document. Retrieval pipelines use this to assess trust and fetch primary evidence rather than relying solely on the docs prose.

Implementation Steps

1. Add source: string to the frontmatter schema (string can be a URL or repo path)
2. For API reference pages: source points to the relevant path in api/openapi.yaml (e.g. source: api/openapi.yaml#/paths/~1stream)
3. For protocol/governance pages: source points to the canonical spec or LIP document URL
4. For SDK pages: source points to the relevant SDK repo file on GitHub
5. Update generate-seo.js to include source in its validation and output; include in docs-index.json
6. For pages with no upstream source (conceptual overviews), use source: self to make the field explicit rather than absent

⚠️ **Caveats**

> *Not every page has a clean upstream source — do not force artificial sources. Conceptual and narrative pages are fine with source: self or source: null. The value is highest on API reference, SDK, and config pages where a primary specification exists. Keep sources as stable URLs, not branch-pinned GitHub links that break on repo restructure.*

Recommended Implementation Sequence

Sprint 1 (1–2 days): Items 1, 3, 4, 5, 11, 19, 23, 25 — file creation, script generation, config, and CI guards. Ship llms.txt, docs-index.json, ai-plugin.json, sitemap-ai.xml, changelog.json, verify OpenAPI URLs live, add HTTP header CI check, and verify robots.txt.

Sprint 2 (3–5 days): Items 7, 9, 10, 14, 20, 24, 26, 27, 28 — frontmatter schema extensions, sitewide metadata rollout, glossary aliases, task map, source field, and promoting the AI quickstart page from x-pages.

Sprint 3 (1 week): Items 6, 8, 12, 22 — content work. Agent runbook section, Q&A blocks on top 10 pages, JSON-LD schema, training-context.md.

Sprint 4 (2 weeks): Items 2, 13, 15, 16, 17, 18 — deeper architectural work. MCP config, concept graph, agent-safety OpenAPI extensions, failure mode pages, .md endpoints.

Sprint 5 (engineering resource): Item 21 — network-state live data endpoint. Requires Cloudflare Worker or serverless infrastructure. Coordinate with Livepeer engineering.